

AutoGen: ne kullanıldı, nerede, neden

Bir VC deal-flow sisteminin AutoGen üstünde kurulmuş hâli — kullanılan her sembol, her modül, ve her tercihin ölçülmüş gerekçesi.

55

AUTOGEN SEMBOLÜ

29

KULLANAN DOSYA

302

GEÇEN TEST

16.650

SATIR KOD

1 · Bu belge ne

Kod tabanı tarandı ve AutoGen'e dokunan her yer çıkarıldı: 29 dosya, 55 farklı sembol. Aşağıda hepsi tek tek, hangi modülde ve *ne için*.

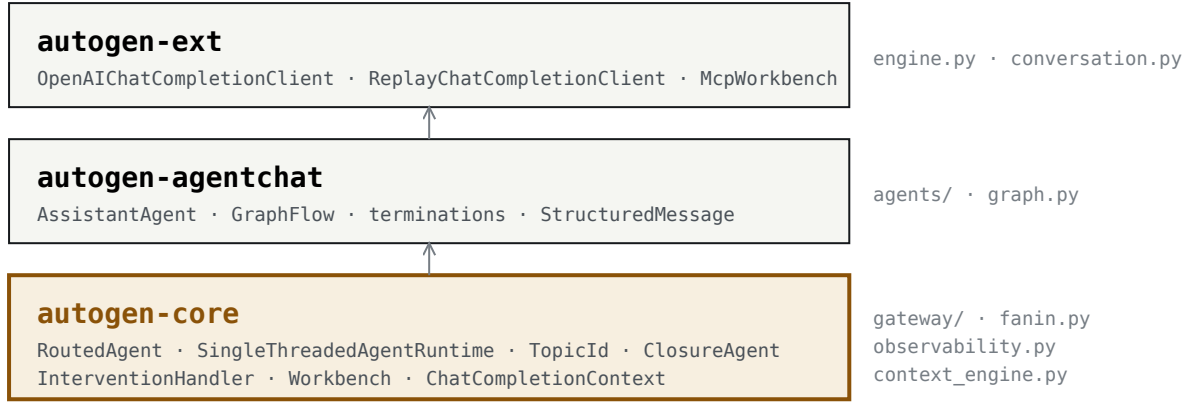
İki şeyi ayrı tutuyorum. **Ne yaptığı** resmî kılavuzda yazıyor ve buraya kopyalanmasının değeri yok. **Neden orada olduğu** ve **çalışırken ne yaptığı** yazmıyor — bu belge onun için.

Her ölçüm etiketli: **ÖLÇÜLDÜ** bu makinede koşturuldu, **KAYNAK** birincil kaynakta yazıyor, **TEYİTSİZ** okundu ama doğrulanmadı.

Sistem bir cümlede

Halka açık kaynakları (Hacker News, SEC Form D, GitHub) tarayıp girişim keşfeden, maliyet gradyanlı bir huniden geçirip skorlayan, ve sonucu bir sohbet arayüzünden sorgulanabilir kılan çok-ajanlı bir sistem. Üstünde OpenClaw mimarisinde bir gateway: oturumlar, hook'lar, onay kapısı, bellek, cron.

2 · Üç katman, ve hangisi nerede



Ağırlık zamanla aşağı kaydı: başta agentchat, sonunda core.

Üç paket ve bu projede hangi dosyaların hangi katmana dayandığı

Proje AgentChat'le başladı — `AssistantAgent`, `GraphFlow`, sonlandırma koşulları. Ölçüm yapıldıkça ağırlık **core**'a kaydı, çünkü ihtiyaç duyulan üç şey yalnız orada vardı:

İHTİYAÇ	CORE'DAKİ KARŞILIĞI	NEREDE
Dal kaybetmeden paralel toplama	<code>ClosureAgent</code> + kuyruk	<code>fanin.py</code>
Maliyet ve tool çağrısı ölçümü	<code>EVENT_LOGGER_NAME</code> olay akışı	<code>observability.py</code>
Teslimden önce durdurabilen kapı	<code>InterventionHandler</code> + <code>DropMessage</code>	<code>observability.py</code>
Oturum başına izole ajan	<code>TopicId</code> kaynağı → ajan anahtarı	<code>gateway/sessions.py</code>
Token sayan bağlam + compaction	<code>ChatCompletionContext</code> alt sınıfı	<code>context_engine.py</code>

3 • Modül modül

29 dosyanın her biri, kullandığı sembollerle. Sıra: önce motor, sonra ajanlar, sonra gateway, sonra ölçüm POC'leri.

3.1 Motor ve ölçüm

pipeline/engine.py

ChatCompletionClient • CreateResult • ModelInfo • RequestUsage • OpenAIChatCompletionClient • ReplayChatCompletionClient

Model fabrikası ve maliyet defteri. Üç kademe (cheap / mid / strong) huninin ekonomisini kodda temsil ediyor: hacim yukarıda ucuz modelde, kalite aşağıda pahalıda.

İki istemci, iki dünya. OpenAIChatCompletionClient canlı; ReplayChatCompletionClient kuru mod — yanıtlar önceden yazılı, ağ yok, deterministik. Bütün test paketi kuru modda koşuyor.

ResilientClient sarmalayıcısı model hatasını *exception değil metin* yapıyor; raw_client ise sarmalamıyor, çünkü sohbetle bir hata cevap gibi görünmemeli.

ÖLÇÜLDÜ model_info OpenAI-uyumlu her endpoint'te **zorunlu**. İlk canlı istekte patladı: ValueError: model_info is required when model name is not a valid OpenAI model . Ve bu bir *beyandır*, ölçüm değil — desteklenmeyen bir yeteneği iddia edersen hata huninin en sonunda, yapısal çıktı üreten skorlayıcıda çıkıyor.

pipeline/observability.py

EVENT_LOGGER_NAME • LLMCallEvent • LLMStreamEndEvent • ToolCallEvent • DefaultInterventionHandler • DropMessage

Maliyet ölçümünün tamamı burada — modele değil, **olay akışına** bakıyor. autogen_core yapılandırılmış olayları ayrı bir logger'a yayıyor; EventCapture bir logging.Handler olarak takılıp sayıyor.

AuditingInterventionHandler ise mesaj yolunda *teslimden önce* oturuyor. DropMessage döndürünce mesaj hiç gitmiyor.

ÖLÇÜLDÜ İki olay, bir anlam. create() → LLMCallEvent; create_stream() → LLMStreamEndEvent ve asla ilki. Sohbetle model_client_stream=True koyunca maliyet 0 raporlamaya başladı. Kod doğruydı; yanlış olayı dinliyordu.

ÖLÇÜLDÜ LLMCallEvent alanlarını öznitelik yapıyor, ToolCallEvent ise hepsini .kwargs sözlüğünde tutuyor. event.tool_name yazınca AttributeError — ve o hata bir logging.Handler içinde olduğu için handleError tarafından **yutuluyor**. Olay hiç kaydedilmiyor, hata da görünmüyor.

Kapının asıl değeri şu cümlede: **onay, ajanın uyum göstermeyi seçmesine değil, runtime'a dayanıyor**. Model ne kadar ikna edilirse edilsin mesaj hattı onun altında.

3.2 Ajanlar – AgentChat katmanı

pipeline/agents/triage.py · analysts.py · memo.py

AssistantAgent · StructuredMessage

Hununin üç kademesi, üç farklı model kademesinde. **Triyaj** en ucuz modelde ve en çok koşan yer; **analistler** (technical / market / team) orta kademede paralel; **not yazarı** en pahalıda ve günde birkaç kez.

output_content_type ile Pydantic şeması dayatılıyor — skorun missing_data alanı **zorunlu**, yani "bakamadım" bilgisi şemadan kaçamıyor.

ÖLÇÜLDÜ output_content_type kullanan bir ajan takımın içindeyse tipi beyan etmen gerekiyor: GraphFlow(..., custom_message_types=[StructuredMessage[Score]]). Tek başına agent.run() 'da gerekmiyor — sorun yalnız *yönlendirmede* çıkıyor: ValueError: Message type StructuredMessage[Score] is not registered.

pipeline/graph.py

GraphFlow · DiGraphBuilder · MaxMessageTermination · TaskResult · StructuredMessage · SingleThreadedAgentRuntime

Paralel zenginleştirme: üç analist dalı fan-out, sonra join. Graf DiGraphBuilder ile çiziliyor, birleşme activation_condition="all".

Runtime **dışarıdan** veriliyor — çünkü InterventionHandler takmanın tek yolu bu. Ve o karar bir yan etki getiriyor:

ÖLÇÜLDÜ Dış runtime verirsən çöken ajan fırlatmıyor, asıyor. Gömülü runtime'da run_stream exception fırlatıyor; dıştan verilende *hiç dönmüyor*. Gelmeyecek bir sonlanma mesajı bekleniyor ve MaxMessageTermination kurtaramıyor, çünkü yeni mesaj da gelmiyor. Duvar saati sınırı burada tedbir değil, **doğruluk şartı**.

pipeline/fanin.py

RoutedAgent · ClosureAgent · ClosureContext · SingleThreadedAgentRuntime · TopicId · TypeSubscription · MessageContext · message_handler

graph.py 'nin core seviyesindeki alternatifi, kılavuzun *Concurrent Agents* deseninden birebir: işçiler sonucu bir topic'e **yayınlıyor**, ClosureAgent bir asyncio.Queue 'ya boşaltıyor.

Fark tek cümlede: **güvenilmeyecek bariyer yok, çünkü bariyer yok**. Dal sonucu ürettiği anda yayınlıyor ve kuyruk onu çoktan tutuyor.

BranchWorker 'ın disiplini bütün projeye yayıldı: *"a failure is published, not raised."*

MOTOR	TEMİZ	SARMALAYICI ARKASINDA	HAM HATA
GraphFlow (AgentChat)	3 dal	2 dal	0-1 dal, süre sınırı dolar
pub/sub + ClosureAgent (core)	3 dal	2 dal	2 dal, ~3 ms

ÖLÇÜLDÜ · aynı arıza enjeksiyonu · pipeline/compare_fanin.py

Kaç dalın kaybolduğu **deterministik değil** — tekrarlı koşullarda 0 ve 1. Ve bundan güçlü bir bulgu: **resmî desenler bu konuda birbiriyle çelişiyor**. *Concurrent Agents* kuyrukla topluyor, *Mixture of Agents* asyncio.gather ile — yani kaybın kaynağı olan yapıyla.

3.3 Sohbet ve bağlam

pipeline/conversation.py

AssistantAgent · FunctionTool · StaticWorkbench · McpWorkbench · StreamableHttpServerParams · CancellationToken

Oturum başına canlı ajan. On dört yerel tool, artı iki uzak MCP kaynağı (DeepWiki, OpenClaw).

model_client_stream=True ile token token akıyor; CancellationToken durdurma düğmesi; save_state/load_state sunucu yeniden başlayınca konuşmayı koruyor.

max_tool_iterations=6 — varsayılan 1, yani zincirleme tool çağrısı yok ve hata da vermiyor, ajan sadece erken susuyor.

ÖLÇÜLDÜ tools= ile workbench= **aynı ajana verilemiyor**: *"Tools cannot be used with a workbench."*

Yerel fonksiyonları StaticWorkbench'e sarıp workbench'e **liste** vermek gerekiyor. Workbench bir tool kaynağı — ajan yazılırken var olmayan tool'ları listeleyebiliyor, ki uzak sunucunun yaptığı tam olarak bu.

pipeline/context_engine.py

ChatCompletionContext · BufferedChatCompletionContext · LLMMessage · SystemMessage · UserMessage · AssistantMessage · FunctionExecutionResultMessage

ChatCompletionContext alt sınıfı. Dört yaşam döngüsü noktası: **Ingest** (add_message) → **Assemble** (get_messages , token bütçesine göre) → **Compact** → **After turn**.

Yazılma sebebi ölçülmüş bir kusur: BufferedChatCompletionContext(24) **mesaj sayıyor, token değil**. Yirmi dört kısa soru ile yirmi dört tarama sonucu ona aynı büyüklükte görünüyor.

Compaction'ı doğru yapan kural: araç çağrısı ile sonucu bir bütün. Bölme noktası bir araç bloğuna düşerse sınır kaydırılıyor — çağrısı özetlenip gitmiş bir FunctionExecutionResultMessage , modelin göremediği bir sorunun cevabı ve sağlayıcılar diziyi reddediyor.

```
def _safe_boundary(messages, index):
    """Kesme noktasını, araç çağrısını sonucundan ayırmayacak yere kaydır."""
    cut = max(0, min(index, len(messages)))
    while cut < len(messages):
        if _is_tool_result(messages[cut]): cut += 1; continue
        if cut > 0 and _is_tool_call(messages[cut-1]): cut += 1; continue
        break
    return cut
```

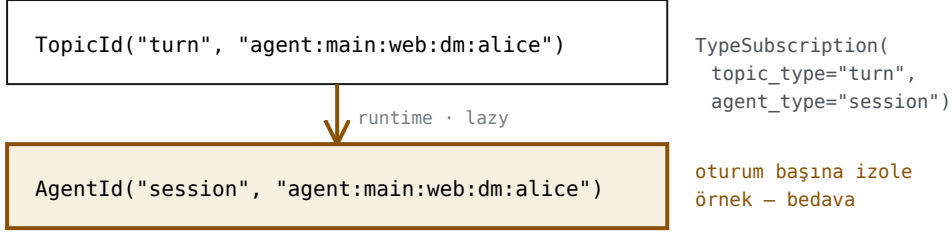
Özetleyici çalışmazsa **kırıyor ve kırptığını söylüyor**: *"...were dropped... They were not summarised, so anything only stated there is no longer available — ask again rather than assuming."*

3.4 Gateway – OpenClaw mimarisi, core motoru

pipeline/gateway/sessions.py

RoutedAgent · TopicId · TypeSubscription · MessageContext · message_handler

Bu projedeki en değerli AutoGen eşlemesi. OpenClaw'ın "oturum anahtarı → izole ajan" modeli, core'un **topic kaynağı → ajan anahtarı** mekanizmasının aynısı.



Oturum kimliği elle tutulan bir sözlük anahtarı değil, runtime'ın kendi adresi

Oturum kimlikleri zaten CloudEvents desenine (`^[\\w\\-\\.\\:\\=]+\\Z`) uyduğu için ara katman gerekmedi. **ÖLÇÜLDÜ** `test_session_agent.py` bunu iddia değil test olarak tutuyor: iki oturum → iki ajan örneği.

pipeline/gateway/runtime.py

`SingleThreadedAgentRuntime`

Süreçteki **tek** runtime, uzun ömürlü. Üç kural, üçü de ölçülmüş bir riske karşı:

1. `stop_when_idle()` *hiç çağrılmıyor* — kaybın kaynağı o bariyer. **2.** `SessionAgent` fırlatmıyor, yayınlıyor. **3.** Transcript modelden önce yazılıyor, yani çöküş cevabı kaybettiriyor, sorunun kaydını değil.

pipeline/gateway/workbench.py

`Workbench · ToolResult · TextResultContent`

Onay kapısının durduğu yer. `InterventionHandler` runtime'ın mesaj yolunda oturuyor; çıplak bir `AssistantAgent` 'ta öyle bir yol yok. `Workbench` ise **her** tool çağrısının geçtiği nokta.

Sarmalayınca kural **ajan yazılırken var olmayan** tool'lar için de geçerli oluyor — bir `workbench`'in tool *kaynağı* olmasının tüm sebebi bu.

Bloklanan çağrı exception değil hata sonucu döndürüyor, çünkü ajanın reddedildiğini söyleyebilmesi gerekiyor.

pipeline/channels.py

`RoutedAgent · TypeSubscription · MessageContext · message_handler`

Her kanal (`web`, `cli`, `openclaw`) bir `ChannelAgent`, `reply.<ad>` topic'ine abone. Bir yüzey eklemek "gönderenin bakacağı sözlüğe kaydet" değil, "topic'e abone et".

3.5 Ölçüm POC'leri

Beş orkestrasyon deseni, **aynı görev, aynı ajanlar** — değişen tek şey kimin konuşacağına nasıl karar verildiği.

poc/desen_1..5 · poc/kiyas.py · poc/motor.py · poc/araclar.py

RoundRobinGroupChat · SelectorGroupChat · Swarm · GraphFlow · DiGraphBuilder · Handoff · RoutedAgent · AgentId · type_subscription · TextMentionTermination · MaxMessageTermination · BaseChatMessage · BaseAgentEvent · ToolCallRequestEvent · ToolCallExecutionEvent · FunctionTool

Bu sekiz dosya projenin *ölçüm zemini*. Desenler kıyaslanabilir olsun diye motor (motor.py) ve tool'lar (araclar.py) ortak.

DESEN	MESAJ	LLM	TOOL	TOKEN	KİMİN KARARI
SelectorGroupChat	8	5	2	204	model — ya da selector_func ile Python
GraphFlow	11	7	3	270	önceden çizilmiş graf
RoundRobinGroupChat	9	6	2	274	sıra — kimseyi atlayamıyor
Swarm	14	7	4	334	ajanın kendisi, devir tool'uyla
autogen_core aktör	9	0	0	0	abonelik

ÖLÇÜLDÜ · poc/kiyas.py · en ucuzla en pahalı arasında %63,7

Ödenen şey zekâ değil, **yönlendirme özerkliği**. Selector yönlendirmeyi bir Python fonksiyonuyla yapıp gereksiz ajanı atlıyor; Swarm'da kararı ajan veriyor ve her devir bir tool çağırısı artı hiç iş üretmeyen bir LLM turu harcıyor.

Bunun framework karşılaştırmasına çevirisi şu: **OpenAI Agents SDK'nın tek modeli olan handoff, AutoGen'in ölçülmüş en pahalı deseni**dir. "Hangi framework daha iyi" sorusu aslında "hangi yönlendirme maliyetini ödemeye razısın" sorusu.

poc/desen_5_core_aktör.py – projenin en önemli bulgusu

"Aktör modeli hata izolasyonu verir" iddiasını üç deneyle test ediyor. Çöken bir handler _process_publish içindeki asyncio.gather'ı erken döndürüyor, hemen ardından task_done() çağırılıyor, kuyruk kardeşler hâlâ çalışırken "boşaldı" sayılıyor.

Tamamlanmış kardeş sonuçlar sessizce kayboluyor — ne exception, ne uyarı. Üç koşuda birebir tekrarlandı.

Aktör modeli runtime'ı koruyor, veriyi korumuyor. Bu cümle projenin bütün mimari kararlarının arkasında duruyor: bariyere güvenme, *beklenen sonuç sayısını say*.

4 · Sembol sözlüğü

55 sembolün tamamı, katmanına göre. Sağ sütun: bu projede ne işe yaradığı.

autogen_core – çekirdek

SEMBOL	BU PROJEDE
<code>RoutedAgent</code>	Tipe göre yönlendiren ajan. Oturum, kanal ve fan-out işçisi.
<code>message_handler</code>	Tip anotasyonundan yönlendirme. <i>Modül düzeyinde import şart.</i>
<code>SingleThreadedAgentRuntime</code>	Gateway'in tek runtime'ı; fan-in ve graf için ayrı örnekler.
<code>TopicId</code>	Yayın adresi. Kaynağı ajan anahtarına dönüşüyor.
<code>TypeSubscription</code>	Topic tipi → ajan tipi. Oturum ve kanal yönlendirmesi.
<code>type_subscription</code>	Dekorator hâli; POC'ta sıralı iş akışında.
<code>AgentId</code>	(tip, anahtar) adresi. POC'ta doğrudan mesajlaşma.
<code>MessageContext</code>	Handler'ın ikinci parametresi. <i>Adı zorunlu: <code>ctx</code>.</i>
<code>ClosureAgent</code> · <code>ClosureContext</code>	Sonuçları kuyruğa boşaltan toplayıcı — fan-in'in kalbi.
<code>DefaultInterventionHandler</code>	Mesaj yolundaki kapı; denetim ve onay.
<code>DropMessage</code>	Teslimi iptal eden dönüş değeri.
<code>EVENT_LOGGER_NAME</code>	Yapılandırılmış olay akışı — maliyet ölçümünün kaynağı.
<code>CancellationToken</code>	Akan bir turu durdurma; <code>/api/chat/stop</code> .
<code>FunctionCall</code>	Tool çağrısı nesnesi; testlerde senaryo kurmak için.

autogen_core.logging · models · tools · model_context

<code>LLMCallEvent</code>	<code>create()</code> 'in yaydığı kullanım olayı.
<code>LLMStreamEndEvent</code>	<code>create_stream()</code> 'in yaydığı — ilkini asla yaymıyor.
<code>ToolCallEvent</code>	Tool çağrısı denetimi. Alanları <code>.kwargs</code> 'ta.
<code>ChatCompletionClient</code>	Model arayüzü; <code>engine.py</code> bunu sarıyor.
<code>CreateResult</code> · <code>RequestUsage</code>	Yanıt ve token sayacı.
<code>ModelInfo</code>	Yetenek beyanı — OpenAI-uyumlu endpoint'te zorunlu.
<code>LLMMessage</code> · <code>SystemMessage</code> · <code>UserMessage</code> · <code>AssistantMessage</code>	Bağlam motorunun taşıdığı mesaj tipleri.
<code>FunctionExecutionResultMessage</code> · <code>FunctionExecutionResult</code>	Tool sonucu — compaction'da çağrısından ayrılmaması gereken yarı.
<code>ChatCompletionContext</code>	Bağlam motorunun taban sınıfı.
<code>BufferedChatCompletionContext</code>	Eski bağlam; artık yalnız fallback.
<code>FunctionTool</code>	Python fonksiyonu → tool. Şema imzadan ve docstring'den.
<code>StaticWorkbench</code>	Yerel fonksiyonları workbench'e sarma — <code>tools=</code> çatışmasının çözümü.
<code>Workbench</code>	Onay kapısının sarmaladığı taban sınıf.
<code>ToolResult</code> · <code>TextResultContent</code>	Reddi <i>sonuç</i> olarak döndürmek için.

autogen_agentchat

AssistantAgent	11 yerde. Triyaj, üç analist, not yazarı, sohbet, POC'ler.
GraphFlow · DiGraphBuilder	Paralel zenginleştirme grafi.
RoundRobinGroupChat · SelectorGroupChat · Swarm	Yalnız POC'ta — ölçüm için.
MaxMessageTermination	Sonsuz döngü sigortası. Her takımda var.
TextMentionTermination	POC'ta "TERMINATE" ile bitirme.
Handoff	POC'ta Swarm devri. <i>Tool adı küçük harfe düşüyor.</i>
StructuredMessage	Pydantic çıktısı; takımda custom_message_types beyanı gerekiyor.
TaskResult	Takım sonucu: messages + stop_reason .
BaseChatMessage · BaseAgentEvent	İletişim/olay ayrımı — arayüzde tool satırlarını ayırmayı sağlıyor.
ToolCallRequestEvent · ToolCallExecutionEvent	Akıшта tool çağrısını göstermek.

autogen_ext

OpenAIChatCompletionClient	Canlı model — base_url ile kendi endpoint'imiz.
ReplayChatCompletionClient	Kuru mod. 302 testin tamamı bunun üstünde.
McpWorkbench	Uzak tool kaynağı — DeepWiki ve OpenClaw.
StdioServerParams	OpenClaw köprüsü (openclaw mcp serve).
StreamableHttpServerParams	DeepWiki bağlantısı.

5 • Ölçülmüş tuzaklar

On üç madde, hepsi **koşarken** bulundu. Ortak örüntü: AutoGen'de arızalar gürültülü değil — exception fırlatmıyor, sıfır dönüyor, boş kalıyor, asılı kalıyor.

#	TUZAK	NASIL GÖRÜNÜYOR
1	<code>tools + workbench</code>	<code>ValueError</code> — ikisi aynı ajana verilemiyor
2	<code>model_context</code> yok	Ajan turlar arası durumsuz; <code>save_state</code> de boş
3	<code>model_info</code> zorunlu	İlk canlı istekte patlıyor; ve bir <i>beyan</i>
4	Akıшта olay değişiyor	Maliyet 0 raporlanıyor
5	<code>create_calls</code> yalnız replay'de	Modlardan birinde ölçüm sıfır, fark edilmiyor
6	<code>ToolCallEvent.kwargs</code>	<code>AttributeError</code> <code>logging.Handler</code> 'da yutuluyor
7	Dış runtime + çöken ajan	<code>run_stream</code> fırlatmıyor, asıyor
8	Paralel dalda çöküş	Kardeşlerin tamamlanmış işi sessizce gidiyor
9	Tanınmayan mesaj tipi	Takım yönlendirmiyor; <code>custom_message_types</code> şart
10	<code>max_tool_iterations=1</code>	Ajan erken susuyor, hata vermiyor
11	<code>@message_handler</code> tip çıkarımı	Kayıtta çıplak <code>NameError</code>
12	<code>mcp>=1.11</code> üst sınırsız	SDK 2.0 çıkınca <code>ImportError</code> , kurulum kırılıyor
13	<code>stop_when_idle()</code>	Bariyer erken açılıyor — güvenme, say

11'in ayrıntısı öğretici. Tip çıkarımı `get_type_hints()` ile yapılıyor ve `from __future__ import annotations` varken anotasyonlar *modül genelinde* çözülüyor. `MessageContext` 'i fonksiyon içinde import edersen kayıt sırasında patlıyor — handler'ın kendisinde değil. Üstelik parametre adları da bağlayıcı: `message` ve `ctx` olmak zorunda.

6 · Kullanılmayanlar, ve neden

Bir envanterin en çok bilgi taşıyan yarısı burası. AutoGen'in kullanmadığımız yüzeyleri, ve her biri için gerekçe.

YÜZEY	NEDEN ALINMADI
<code>GrpcWorkerAgentRuntime</code> — dağıtık runtime	<code>grpcio</code> kurulu değil; tek süreç yetiyor. Hacim sorusu cevaplanınca ilk bakılacak yer.
<code>dump_component</code> / <code>load_component</code>	Bildirimsel ajan tanımına ihtiyaç doğmadı; her şey kodda.
<code>CodeExecutorAgent</code> , kod yürütücüler	VC hattında kod çalıştırılmıyor. AutoGen'in en güçlü olduğu eksen, bize kapalı.
<code>Memory</code> protokolü (ChromaDB, mem0)	Bellek düz Markdown yapıldı — okunabilir, düzeltilebilir, sürüm kontrolüne girebilir.
<code>MagenticOneGroupChat</code>	Genel amaçlı orkestratör; huninin şekli zaten belli.
<code>Swarm</code> / <code>Handoff</code> (pipeline'da)	POC'ta ölçüldü: en pahalı desen . Huni tek yönlü, devre ihtiyaç yok.
<code>SocietyOfMindAgent</code>	Takım iç içe geçirme gerekmedi — ama hiyerarşi lazım olursa hazır.
<code>OpenTelemetry</code> (<code>tracer_provider</code>)	Kalan işlerin en kolayı: runtime'ı zaten kendimiz kuruyoruz, tek satır.
<code>TokenLimitedChatCompletionContext</code>	Kendi bağlam motorumuz yazıldı, çünkü compaction'da tool çifti kuralı gerekti.
<code>Sandbox</code>	Model üretimi kod çalıştırmıyoruz.

7 · Üç cümlede özet

1 · AutoGen'i ayıran şey üst katman değil, alt katman

`AssistantAgent` her framework'te var. `autogen_core` karşılığı çoğunda yok — ve bu projenin işine yarayan her şey (izole oturum, olay akışı, müdahale kapısı, kuyrukla toplama) orada.

2 · Aktör modeli runtime'ı koruyor, veriyi korumuyor

Ölçülmüş, tekrarlanmış, ve resmî desenlerin kendi arasında çeliştiği nokta. Bariyere güvenilmiyor; **beklenen sonuç sayısı sayılıyor**.

3 · Taşınabilir olan şey desenler ve protokoller

AutoGen bakım modunda. Ama CloudEvents, MCP, OTEL ve dokuz tasarım deseni onun *dışında* yaşıyor. Bu yüzden AutoGen burada ürün olarak değil, **desen kütüphanesi ve ölçüm zemini** olarak kullanıldı.

8 · Bu belgede ölçülmemiş olanlar

- Dağıtık runtime hiç çalıştırılmadı.** `grpcio` yok; protokol bilgisi `.proto` descriptor'ından okundu — arayüz bilgisi, davranış değil.
- Diğer framework'lerin fan-in davranışı ölçülmedi.** Karşılaştırma yalnız AutoGen'in iki motorunu kapsıyor; "LangGraph daha iyi olurdu" diyemem, denemedim.
- Token ölçümleri tek koşudan.** `poc/kiyas.py` tekrar çalıştırılabilir ama sayılar tek örneklem; model sıcaklığı ve endpoint yükü etkiliyor.

- **Compaction canlı modelle denenmedi.** Kuru modda kırpma yolu ve tool-çifti kuralı testli; `model_summariser` uzun bir gerçek sohbette koşmadı.
- **OTel dışı aktarımı kurulmadı.** `gen_ai.*` sabitlerinin kodda olduğu görüldü; gerçek bir backend'e span gittiği görülmedi.

vc-agent · pipeline 15.589 satır · poc 1.061 satır · docs 27.232 satır · 302 test

Kaynak belgeler: `docs/05` ve `docs/08` (resmî kılavuzların tam metni) · `docs/06` (ölçülmüş tuzaklar) · `docs/07` (kod rehberi) · `docs/12-15` (uçtan uca anlatım, protokoller, gateway mimarisi)